# MEA(R)N Full Stack Programming

# Objectives

- **In this session, you will learn to....**
  - Introduction to NoSQL
  - Advantages and Disadvantages of NoSQL Database
  - Introduction to MongoDB
  - Architecture of MongoDB
  - Components of MongoDB
  - Features of MongoDB
  - Why Use MongoDB ?
  - Difference between NoSQL vs RDBMS

# NoSQL Database

- NoSQL databases are non-tabular and store data in formats other than traditional relational tables.
- NoSQL databases are categorized into different types based on their underlying data models such as:
    1. **Document database -** They store data in JSON-like documents that can include various data types such as strings, numbers, booleans, arrays, and objects.
    2. **Key-Value database -** each item contains keys and values.
    3. **Wide-Column database -** store data in tables, rows, and dynamic columns
    4. **Graph database -** store data in nodes and edge, where node contains data and edge store the relationships between nodes.

## Key-value pair

| Key | Value |
|---|---|
| Name | Joe Bloggs |
| Age | 42 |
| Occupation | Stunt Double |
| Height | 175cm |
| Weight | 77kg |

## Document Based

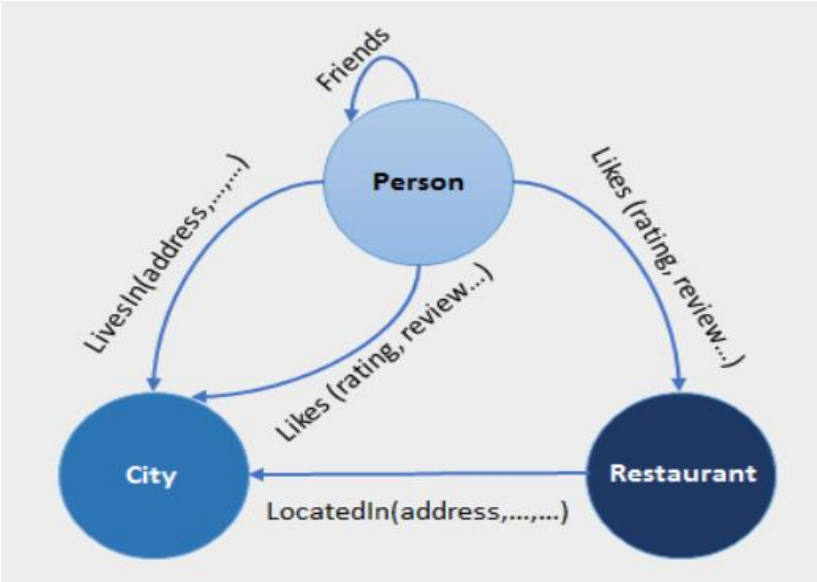
**Document 1**

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

**Document 2**

```
{
  "prop1": data,
  "prop2": data,
  "prop3": data,
  "prop4": data
}
```

## Graph based




## Column based

| ColumnFamily | | | |
|---|---|---|---|
| Row Key | Column Name | | |
| | Key | Key | Key |
| | Value | Value | Value |
| | Column Name | | |
| | Key | Key | Key |
| | Value | Value | Value |

# Advantages of NoSQL Database

- NoSQL databases streamline application development, especially for interactive, real-time web applications that utilize REST APIs and web services.
- It offers flexibility for unnormalized data that requires a dynamic data model or contains varying properties across different data entities.
- It provides scalability to handle large data sets, which are often used in analytics and artificial intelligence (AI) applications.
- NoSQL databases are well-suited for modern cloud, mobile, social media, and big data applications.
- It is designed for specific use cases and is easier to use than general-purpose relational or SQL databases for those applications.

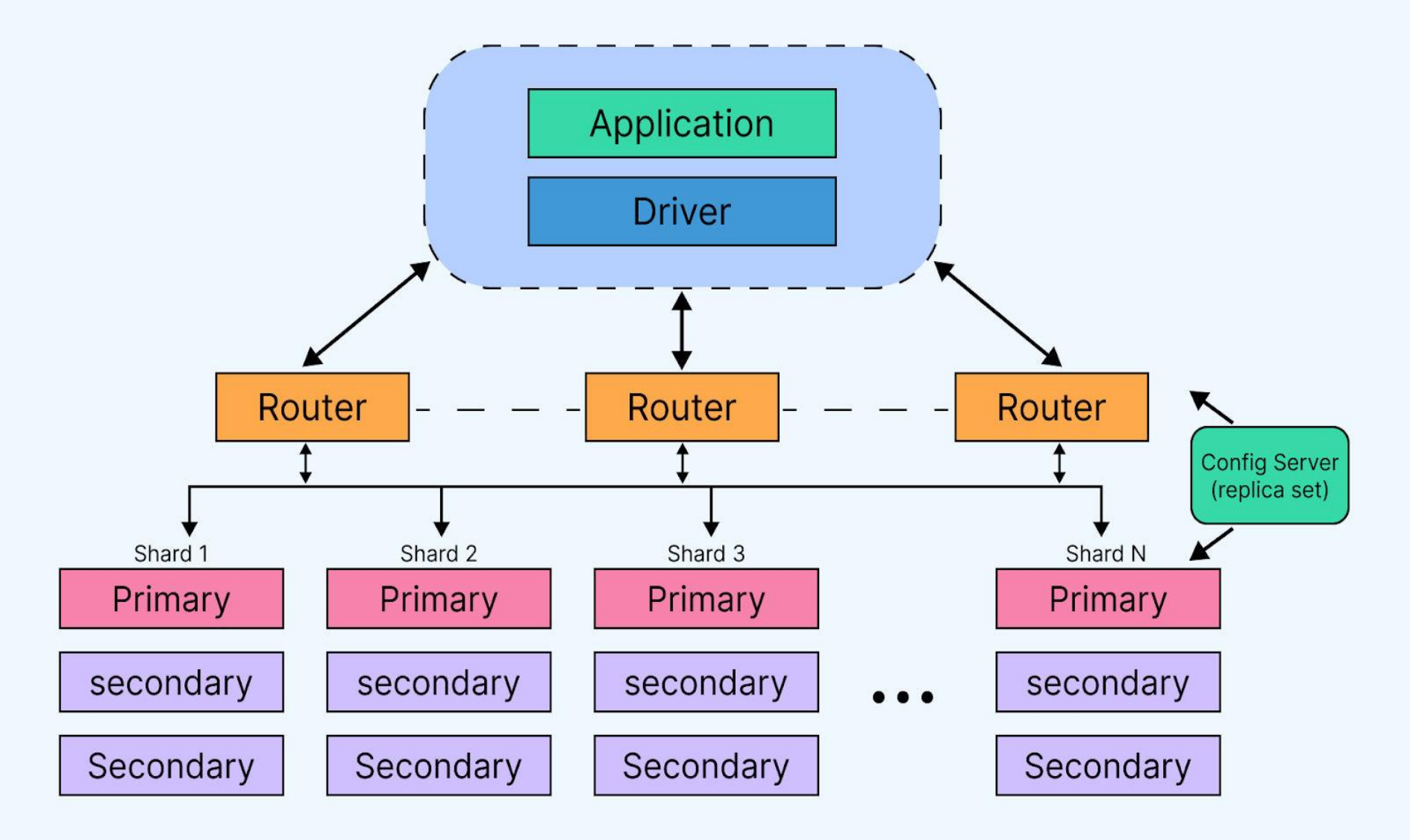
# Disadvantages of NoSQL Database

- Lacks standardization rules
- Limited query capabilities
- Does not provide traditional database features like full consistency during multiple simultaneous transactions.
- Maintaining unique keys becomes challenging as data volume grows.
- Not well-suited for handling relational data.
- Being mostly open-source, it is less commonly adopted by large enterprises.

# What is MongoDB ?

- MongoDB is a scalable, open-source, high-performance, document-oriented NoSQL database.
- Its data model supports representing hierarchical relationships.
- It stores data in JSON-like documents with an optional schema, instead of traditional tables and rows.
- Documents made up of key-value pairs serve as the basic data units in MongoDB.
- It was originally developed by **10gen**, which was later renamed **MongoDB Inc.**

# Architecture of MongoDB



# Let’s understand each Components of MongoDB Architecture

***Shared***

- Shards are individual database instances in MongoDB's architecture.
- It store subsets of the database, with each handling specific chunks of data.

***Server***

- A MongoDB server is where the database instances run.
- It includes a primary server that handles write operations and secondary servers that replicate data for availability.

***MongoDB Engine***

- It processes all data operations, such as reads, writes, and updates.
- It includes features like query execution, indexing, and aggregation frameworks to optimize data retrieval and processing.

# Let’s understand each Components of MongoDB Architecture

## *Config Server*

- Config servers contain metadata about the database's structure and sharding information.
- It track which shard holds specific data and route client queries to the correct shard.

## *Mongos / Router*

- Mongos is a query router that directs client requests to the appropriate shard.
- It acts as an intermediary between the application and the sharded database.

## *Replica Set*

- A replica set is a group of servers that maintain the same dataset.
- It includes a primary node for write operations and multiple secondary nodes for handling failure..

## Features of MongoDB ( Contd. )

***Schema-less Database***

- One collection can hold different types of documents in terms of fields, contents, and size.
- It is not necessary that the one document is similar to another document like in the relational databases.

***Document oriented***

- Data is stored in fields ( key-value pairs ) instead of rows and column.
- Each document contains unique object ID.

***Indexing***

- Every field in the documents is indexed with primary and secondary indices this makes easier and takes less to get or search data from the pool of data.

## Features of MongoDB

### *Scalability*

- It provides horizontal scalability with the help of sharding.
- large amount of data is partitioned into data chunks using the shard key, and these data chunks are evenly distributed across shards that reside across many physical servers.

### *Replication*

- high availability and redundancy with the help of replication.
- it creates multiple copies of the data and sends these copies to a different server so that if one server fails, then the data is retrieved from another server.

### *Aggregation*

- It allows to perform operations on the grouped data and get a single result or computed result.

## Why use MongoDB ?

- It is build on a ***scale-out-architecture*** that has become popular with developers of all kinds for developing scalable applications with evolving data schemas.
- It easy for developers to store ***structured*** and ***unstructured*** data.
- It used ***JSON*** format which is directly maps to native objects in most modern programming languages.
- No need to think about ***normalizing*** data.
- It can scale both ***vertically*** or ***horizontally*** to accommodate large data loads.
- It is widely used in various industries such as ***telecommunications, gaming, finances, healthcare, and retail etc.***

# How MongoDB is different from RDBMS ?

| NoSQL | RDBMS |
|---|---|
| It is a non-relational and document-oriented database. | It is a relational database. |
| It is suitable for hierarchical data storage. | It is not suitable for hierarchical data storage. |
| It has a dynamic schema. | It has a predefined schema. |
| It centers around the CAP theorem (Consistency, Availability, and Partition tolerance). | It centers around ACID properties (Atomicity, Consistency, Isolation, and Durability). |
| Much faster than RDBMS. | It slower than NoSQL database. |

# Install and Configuration of MongoDB

# Objectives

- **In this session, you will learn to....**
  - Introduce to Data Modeling and Schema Design
  - Working with databases
  - Working with collections
  - Working with primitive datatypes
  - Working with query
  - CRUD operations with collection
  - Working with operators
  - Working with collection methods

# Data Modeling and Schema Design

## What is Data Modeling ?

- Data modeling serves as the blueprint for developing a complete database system.
- The main function of a data model is to facilitate visual information of how the relationship between two or a group of data points.
- Data models in a database system can generally be classified into three categories:

1. Conceptual Data Models
2. Logical Data Models
3. Physical Data Models

# Data Modeling and Schema Design

1. ***Conceptual data models***
   - It contains the entity class, characteristics, constraints, and the relationship between security and data integrity requirements.
2. ***Logical data models***
   - It provide subjective information on the relationships between data sets
3. **Physical data models**
   - A physical data model defines the schema and structure for how data is stored within a database.
   - It provides a finalized design that can be directly implemented in a relational database system.

# Types of Data Model

1. Embedded data model
2. Normalized data model

## Embedded Model

```
{
  _id: <ObjectId1>,
  username: "123xyz",
  contact: {
              phone: "123-456-7890",
              email: "xyz@example.com"
            },
  access: {
             level: 5,
             group: "dev"
           }
}
```

Embedded sub-document

Embedded sub-document

## Normalized Model

user document

```
{
  _id: <ObjectId1>,
  username: "123xyz"
}
```

contact document

```
{
  _id: <ObjectId2>,
  user_id: <ObjectId1>,
  phone: "123-456-7890",
  email: "xyz@example.com"
}
```

access document

```
{
  _id: <ObjectId3>,
  user_id: <ObjectId1>,
  level: 5,
  group: "dev"
}
```

# Relationship in MongoDB Data Modeling

## 1. One-to-One Relationship

- Represents a single connection between two entities.
- Best implemented using **embedding**, as the related data is small and tightly coupled.
- Example: One **User** has one **Profile**.

**Embedded Example:**

```
{
    "user_id": 1, "name": "Alice",
    "profile": {
        “email”: “alice@example.com”, "gender": “F", "city": "Delhi“ }
}
```

## 1. One-to-One Relationship

**Reference Example:**

```
db.profiles.insertOne({
    “email”: “alice@example.com”,   "gender": "Female", "city": "Delhi"
})
```

After inserting, note the returned **_id**: ObjectId("**675abcd12345ef6789012345**")

```
db.users.insertOne({
 "user_id": 1,
 "name": "Alice",
 "profile":  ObjectId("675abcd12345ef6789012345")
}
```

# Relationship in MongoDB Data Modeling

## 2. One-to-many Relationship

- Represents multiple entities linked to multiple others.
- Can be modeled using **embedding** (for small datasets) or **referencing** (for large datasets).
- Example: One **customer** can have multiple **orders**.

## Embedded Example:

```
{
    name: "Rohit Sharma", email: "rohit@aol.com",
   "orders": [ {product: "Laptop", price: 55000 },
       { product: "Keyboard", price: 1999 } ]
}
```

## 2. One-to-many Relationship

**Reference Example:**

```
db.users.insertOne( { name: "Rohit Sharma", email: "rohit@aol.com" })
```

After inserting, note the returned **_id**: ObjectId("**675abc123def4560001112 22**")

```
db.orders.insertMany([
  { product: "Laptop", price: 55000,
        userId: ObjectId("675abc123def4560001112 22") },
  { product: "Keyboard", price: 1999,
        userId: ObjectId("675abc123def4560001112 22") }
])
```

# Relationship in MongoDB Data Modeling

## 3. Many to Many Relationship

- Represents multiple entities linked to multiple others.
- Usually implemented using **referencing** with arrays of IDs in both collections.
- **Example:** A student can enroll in multiple courses, and each course can have multiple students.

**# Students collection**

```
{
  "student_id": 1,
  "name": "Sara",
  "courses": [101, 102]
}
```

**# Courses collection**

```
{
  "_id": 101,
  "name": “Java Programming",
  “enrolledStudents": [1, 2, 3]
}
```

## 3. Many to Many Relationship

**Reference Example**

```
db.users.insertMany( [ { name: "Maths" }, { name: "Physics" } ] )
```

After inserting, note the returned **_id**:

**Maths** → ObjectId("**675aaa111bbb222ccc333111**")

**Physics** → ObjectId("**675aaa111bbb222ccc333222**")

```
db.students.insertMany( [ {
    name: "Amit", courses: [
        ObjectId("675aaa111bbb222ccc333111"),
        ObjectId("675aaa111bbb222ccc333222") ]
  }, { name: "Sara", courses: [ ObjectId("675aaa111bbb222ccc333111") ]}
])
```

# Creating Database

- MongoDB does not provide command to create a new database.
- Instead, you can use the **use** command followed by the desired database name to create a new database.
- If a database with that name already exists, MongoDB will switch to it; otherwise, it will create a new one automatically.

**Syntax :** use **<database** name**>**

**Example:**

use myDatabase

# Database Operation Commands

- To display all existing databases, use the following command.
  **show databases**
  or
  **show dbs**

- To view the currently selected database, use the following command.
  **db**

- To delete an existing database, use the following command.
  **db.dropDatabase()**

# Collection Commands

- Collections in MongoDB are similar to tables in relational databases; however, they store data in the form of documents.
- A single database can contain multiple collections.
- To display all collections in a database, use the following command:

  ***show collections*** **Or** ***show tables***

- To create a new collection

  ***Syntax :*** *db.createCollection(<collection name>)*

  db.createCollection("employees")

# Collection Commands

- To rename an existing collection, execute the following command.

  ***Syntax :***

  db.<collection_name>.renameCollection(<new_name>)

  db.employee.renameCollection("employees")

- To delete an existing collection from the database, use the following command.

  ***Syntax :*** db.<collection_name>.drop()

  db.employee.drop()

# General Datatypes in MongoDB

1. **String -** Stores text data. It is the most commonly used data type in MongoDB.
   **Example :** { "name": "John Doe" }

2. **Integer (int, long) -** Stores numeric (whole number) values. Used for counting, indexing, etc.
   **Example :** { "age": 25 }

3. **Double -** Stores floating-point numbers (numbers with decimals).
   **Example :** { "price": 99.99 }

# General Datatypes in MongoDB

4. **Boolean -** Stores true or false values.
   **Example :** { "isActive": true }

5. **Array -** Stores multiple values in a single key (like a list).
   **Example :** { "skills": ["JavaScript", "Node.js", "MongoDB"] }

6. **Object / Embedded document -** Stores a document inside another document (used for nested data).
   **Example :** { "address": { "city": "Surat", "pincode": 395007 } }

7. **Null -** Represents a null or missing value.
   **Example :** { "middleName": null }

# General Datatypes in MongoDB

8. **ObjectId -** A unique identifier automatically generated for each document in a collection.
   **Example :** { "name": "John Doe" }

9. **Date :** Stores date and time in ISODate format.
   **Example :** { "createdAt": ISODate("2025-11-06T10:00:00Z") }

10. **Binary Data -** Used to store binary data like images or files.
    **Example :** { "fileData": <binary> }

# General Datatypes in MongoDB

11. **Regular Expression –** Stores regular expression for pattern matching.
    **Example** : { "email": /@gmail\.com$/ }

12. **Timestamp :** Stores the current time in a special timestamp format Commonly used for internal MongoDB operations
    **Example :** { "lastModified": Timestamp(1633024800, 1) }

13. **Decimal128 -** Stores high-precision decimal values (useful in financial applications).
    **Example :** { "salary": NumberDecimal("12345.67") }

# CRUD Operations

# Insert Document

- The insert() method is used to add one or more documents into a collection.
- You can insert documents with or without specifying an **_id** field, as MongoDB automatically generates an **_id** for each document if it's not provided.
- *db.Collection_name.insert(<document or [ document1, document2…]>)*
- **Examples:**

1. db.employees.insert(
   {"id": "E001", "fullname": "Sanjay Shah", "Mobile": "9998321992"})

# Insert Document

2. document = ( {"id": "E002", "fullname": "Mahesh Patel", "Mobile": "9824024212"} )

   db.employees.**insert**(document)

3. db.employees.**insertOne**( {"id": "E003", "fullname": "Aishwarya Rai", "Mobile": "9824452525"})

4. db.users.**insertOne**({

   name: "Kalpesh", email: "kalpesh@example.com",

   address: { street: "123 Main St", city: "Surat" }

   })

## Insert Documents with arrays:

```
5. db.products.insertOne({
        name: "Laptop", price: 999,
        tags: ["electronics", "computers", "portable"],
        specifications:
        {
            processor: "Intel i7", ram: "16GB",  storage: "512GB SSD"
        },
        createdAt: new Date()
})
```

## Insert multiple Documents :

```
6. db.products.insertMany( [
    { name: "Laptop", price: 999, category: "Electronics" },
    { name: "Mouse", price: 25, category: "Electronics" },
    { name: "Keyboard", price: 75, category: "Electronics" }
] )
```

## Inserting documents with varying structures:

```
db.items.insertMany( [
  { name: "Item A", price: 100 },
  { name: "Item B", price: 200, discount: 10 },
  { name: "Item C", price: 150, inStock: true, tags: ["sale"] }
] )
```

# Insert Document ( Bulk Insert Operations )

```
var bulk = db.stud.initializeOrderedBulkOp()

bulk.insert( { first_name: "Jalpa", last_name: "Shah" } )
bulk.insert( { first_name: "Mahesh", last_name: "Bhatt" } )
bulk.insert( { first_name: "Naresh", last_name: "Vagashsia" } )
bulk.insert( { first_name: "Prakash", last_name: "Patel" } )
bulk.insert( { first_name: "Pinal", last_name: "Srivasta" } )

bulk.execute();
```

# Insert large batches of data – Example-1

```
for(let i=0; i<=100; i++) {
    db.users.insertOne({
        username: `user${i}`,
        email: `user${i}@example.com`,
        createdAt: new Date()
    })
}
```

# Insert large batches of data – Example-2

```
const users=[]
for(let i=0; i<=100; i++) {
    users.push({
        username: `user${i}`,
        email: `user${i}@example.com`,
        createdAt: new Date()
    })
}
db.users.insertMany(users);
```

# Update Document

- The **update()** method is used to modify the values of existing documents within a MongoDB collection.
- *db.collection_name.update({selection_criteria},*

  *{$set:{updated_data}},*

  *{ upsert: <boolean>, multi: <boolean> })*

- The *{*selection_criteria*}* is the older value in the form of documents.
- The second parameter must contain a $set keyword to update the following specify document values.
- **upsert** – if set to ***true***, create a new document if no documents matches the query.
- **multi** – if set the ***true***, update multiple documents that match the query.

# Update Document

**Examples:**

- db.students.update(
  {"rollno":"200300999011"},
  {$set: {"first_name":"Mahesh","last_name":"Shah"}})
- db.students.updateOne({"city":"Surat"},{$set: {"city":"Nasari"}})
- Phoneno fields does not exists so it add as new fields into collection
  db.students.update({"rollno":"200300999011"},{$set:
  {"phoneno":"400300125"}})
- db.students.updateMany({"city":"Anand"},{$set: {"country":"india"}})

# Update Document

```
db.users.updateOne(
    { name: "John Doe" },
    { $set: { "address.city": "Boston" } }
)

//Updating all documents in the collection
db.products.updateMany(
    {},
    { $set: { reviewed: false } }
)
```

## Operators with Update Document

**$inc** – this operator increments a numeric field by the specified value. Negative values decrement.

```
db.users.updateOne(
    { username: "john" }, { $inc: { loginCount: 1, points: 10 } }
)

db.products.updateOne(
    { name: "Laptop" }, { $inc: { stock: -1 } }
)
```

# Operators with Update Document

**$push** – $push operator adds an element to an array. If the array doesn't exist, it creates it.

```
db.users.updateOne(
    { name: “Akash" }, { $push: { hobbies: “Watching News" } }
)

db.students.updateOne( { _id: 1 },
    {  $push: {
        subjects: { $each: ["English", "History"] }
    }
 })   //$each with $push adds multiple elements to the array
```

# Operators with Update Document

**$pull** – $pull operator removes elements from an array that match the specified condition.

```
db.products.updateOne(
    { _id: 1 },
    { $pull: { reviews: { rating: { $lt: 3 } } } }
)
```

**//This removes all reviews with ratings less than 3 from the reviews array.**

# Operators with Update Document

## $unset

- It is used to remove a field from a document.
- It does NOT delete the document, only removes specific field(s).

```
db.users.updateOne(
    { name: “akash" }, { $unset: { hobbies: "" } }
)
```

# Operators with Update Document

## $addToSet

- It is used to add a value to an array only if it does NOT already exist.
- It ensures no duplicates are inserted into the array — similar to Set in programming.

```
db.users.updateOne(
    { _id: 1 },
    { $addToSet: { skills: "Node.js" } }
);
```

# Operators with Update Document

## $pop

- used to remove the first or last element from an array field.
- It works like pop operations in typical data structures.

```
{
 "_id": 1, "numbers": [10, 20, 30, 40, 50]
}

db.test.updateOne(
    { _id: 1 }, { $pop: { numbers: 1 } }    //It remove the 1 element -> 10
);
```

# Operators with Update Document

## $uncert

- Upsert operations combine insert and update logic: if a document matches the filter, it's updated; if not, a new document is inserted.

```
db.users.updateOne(
    { email: "new@example.com" },
    {  $set: {  name: "New User", email: "new@example.com" },
    { upsert: true }
)
```

# Replace

**replaceOne()** method is used to replace an existing document with a new document.

**Syntax :** *db.collection.replaceOne(<filter>, <replacementDocument>)*

```
db.stud.replaceOne(
        {},
        {"first_name": "Anna", "last_name":"khan", "city":"Delhi"})

db.students.replaceOne(
        { rollno: "200300999010" },
        {first_name: "Ramesh",  last_name: "Yadav", city: "Vadodara",
        course: "Advance Java" } )
```

# Delete Document

- The remove() method deletes a documents from a database collection.
- It can remove a single document or multiple documents that match the specified query criteria.
- If an empty document {} is passed to this method, it will delete all documents within the specified collection.
- **Syntax** : *db.Collection_name.remove(<matching_criteria>)*

# Delete Document

**Examples:**

- db.students.remove( {} )
- db.students.deleteOne( {} )

- db.students.remove ({ "first_name": "Ganesh" }, true)
- db.students.deleteOne({ "first_name": "Ganesh" })

- db.students.deleteMany({})
- db.students.deleteMany( { city: { $in: [ “Anand” ] } } )

# Delete Document

```
const user = db.users.findOne({ email: "user@example.com" })
if (user) {
    print("Found user: " + user.name)
    db.users.deleteOne({ _id: user._id })
}
else
{
    print("No user found with that email")
}
```

## Validation Errors

A collection can be created with a JSON Schema validator that enforces structure, required fields, and data types.

```
db.createCollection("products", {
    validator: {
        $jsonSchema: { required: ["name", "price"] }
    }
);

db.products.insertOne({ name: "Laptop" }); // Validation Error
```

# Querying Document

**Find () –** This method retrieves multiple documents from a collection and returns a cursor object

## Examples:

## To list all the documents from employee collection.

db.employees.find()

## To list all the documents in organized form.

db.employees.find().pretty()

## To list only specific fields from collection.

db.employees.find({}, {fullname: 1, Mobile: 1} )

db.employees.find({}, {fullname: true, Mobile: true} )

**To list all the documents based on criteria from collection.**

- db.employees.find()
- db.employees.find({}, {**“Mobile”:”9998321992”**})
  db.employees.find({“Mobile”:”9998321992”})
- db.employees.find().pretty();
- db.products.find({ “category”: “Electronics”}, {“name”:1, “price”: 1, _id: 0})
- db.students.find({
  "city":"Anand",
  "course":"Full Stack using Python"}).pretty();  **// implicit AND logic**

**To list all the documents based on criteria from collection.**

- db.users.find({“address.city”: “Vadodara”})
- db.orders.find({ "customer.contact.phone": "9876543210" })
- db.users.find({}, { password: 0, role: 0 }) **//all except these two fields**
- let result = db.employees.find().toArray()
  print ( result )
  result.foreach((e) => print(“Full name :”, e.fullname)
- db.users.countDocuments({})
  db.users.countDocuments({ userType: “admin”})

# Querying Document ( using $and, $or, $not, $nor )

**$and -** is a **logical operator** used to combine multiple query conditions. It returns the documents that satisfy **all** the specified conditions within the array.

**Syntax :** *{ $and: [ { Expression1 }, { Expression2 }, ..., { ExpressionN } ] } or*
*{ { Expression1 }, { Expression2 }, ..., { ExpressionN }}*

**Examples:**

- db.students.find( {$and: [{"city":"Anand"}, {"course":"FS using Python"}]}).pretty()
- db.students.find({"city":"Anand", "course":"FS using Python"}).pretty()
- db.students.find({"city":"Anand","course":"FS using Python"},
  {"first_name":1, "_id":false}).pretty()

# Querying Document ( using $and, $or, $not, $nor )

**$or** - is a **logical operator** used to combine multiple query conditions, returning documents that satisfy **at least one** of the specified conditions.

**Syntax :** *{ $or: [ { Expression1 }, { Expression2 }, …, { ExpressionN } ] }*

**Examples:**

- db.students.find( {$or: [{"city":"Anand"}, {"city": “Bharuch"}]}).pretty()

- db.students.find({$or: [{"city":"Anand"}, {"city ": "Bharuch"}]},
  {"first_name":1, “last_name”:1}).pretty()

# Querying Document ( using $and, $or, $not, $nor )

**$not -** is a **logical negation operator** that **inverts** the effect of a query expression. It returns documents that **do not match** the specified condition.

**Syntax :** *{ field: { $not: { operator-expression } } }*

**Examples:**

- db.students.find({"city": {$not: {$eq : "Anand"}}},
  {"first_name":1,"city":1}).pretty()

- db.students.find({"city": {$not: {$in :
  ["Anand","Vadodara","Surat"]}}},
  {"first_name":1,"city":1}).pretty()

# Querying Document ( using $and, $or, $not, $nor )

**$nor -** is a **logical operator** that joins multiple query expressions using. It returns **only those documents** that **fail all** of the specified conditions meaning **none** of the conditions inside **$nor** should be true.

**Syntax :** *{ $nor: [ { Expression1 }, { Expression2 }, … { ExpressionN } ] }*

- db.students.find({$nor:
  [ {"city": "Anand"},{"first_name":"Ganesh"} ] },
  {"first_name":1,"city":1}).pretty()

- db.students.find({$nor: [
  {"city": {$in: ["Anand", "Vadodara"]}}
  ]}, {"first_name":1,"city":1}).pretty()

# Querying Document ( Using Comparison Operators)

**$gt -** Matches values **greater than** a specified value.

**Syntax :** *{field: {$gt: value}}*

- db.emps.find({"salary": {$gt: 15000}}).pretty();
- db.emps.find({$and: [{"salary": {$gt: 15000}}, {"gender": "male"}]}).pretty();
- db.emps.find({$and: [{"salary": {$gt: 15000}}, {"gender": "male"}, {"age": {$gt: 38}}]}).pretty();
- db.employee.deleteMany({
  salary: { $lte: 40000}
  })

# Querying Document ( Using Comparison Operators)

**$lt -** Matches values **less than** a specified value.

**Syntax** *: {field: {$lt: value}}*

- db.emps.find({"salary": {$lt: 15000}}).pretty();
- db.emps.find({$and: [{"salary": {$lt: 15000}}, {"gender": "male"}]}).pretty();
- db.emps.find({$and: [{"salary": {$lt: 15000}}, {"gender": "male"}, {"age": {$lt: 38}}]}).pretty();
- db.employee.updateOne(
  { salary: { $lte: 40000 } }, { $set: { lastActivity: new Date() } } )

# Querying Document ( Using Comparison Operators)

**$eq -** Matches values **equal to** a specified value.

**Syntax :** *{field: {$eq: value}}*

- db.emps.find({"salary": {$eq: 15000}}).pretty();
- db.emps.find({$and: [{"salary": {$eq: 15000}}, {"gender": “female"}]}).pretty();
- db.emps.find({$and: [{"salary": {$eq: 15000}}, {"gender": "male"}, {“job_title": {$eq: “DevOps Engineer”}}]}).pretty();

# Querying Document ( Using Comparison Operators)

**$gte -** Matches values **greater than or equal to** a specified value.

**Syntax :** *{field: {$gte: value}}*

- db.emps.find({"salary": {$gte: 15000}}).pretty();
- db.emps.find({$and: [{"salary": {$gte: 15000}}, {"gender": "male"}]}).pretty();
- db.emps.find({$and: [{"salary": {$gte: 15000}}, {"gender": "male"}, {"age": {$gte: 38}}]}).pretty();

# Querying Document ( Using Comparison Operators)

**$lte -** Matches values **less than or equal to** a specified value.

**Syntax :** {field: {$lte: value}}

- db.emps.find({"salary": {$lte: 15000}}).pretty();
- db.emps.find({$and: [{"salary": {$gte: 15000}}, {"gender": "male"}]}).pretty();
- db.emps.find({$and: [{"salary": {$gte: 15000}}, {"gender": "male"}, {"age": {$gte: 38}}]}).pretty();

# Querying Document ( Using Comparison Operators)

**$ne -** Matches values **not equal to** a specified value.

**Syntax :** *{field: {$ne: value}}*

- db.emps.find({"salary": {$ne: 15000}}).pretty();
- db.emps.find({$and: [{"salary": {$ne: 15000}}, {"gender": “female"}]}).pretty();
- db.emps.find({$and: [{"salary": {$ne: 15000}}, {"gender": "male"}, {"age": {$ne: 38}}]}).pretty();

# Querying Document ( Using Comparison Operators)

**$in -** Matches values **within** an array of possible values.

**Syntax :** *{field: {$in: [value1, value2, value3, ...]}}*

- db.emps.find({"job_title": {$in: ["Tester","Designer"]}}).pretty()
- db.emps.find({$and: [{"job_title": {$in: ["Tester","Designer"]}}, {"salary": {$gt: 15000}}]}).pretty()
- db.emps.find({$and: [{"job_title": {$in: ["Tester","Designer"]}}, {"age": {$in: [41,32]}}]}).pretty()

# Querying Document ( Using Comparison Operators)

**$nin -** Matches values **not in** the specified array.

**Syntax :** *{field: {$nin: [value1, value2, value3, ...]}}*

- db.emps.find({"job_title": {$nin: ["Tester","Designer"]}}).pretty()
- db.emps.find({$and: [{"job_title": {$nin: ["Tester","Designer"]}}, {"salary": {$gt: 15000}}]}).pretty()
- db.emps.find({$and: [{"job_title": {$nin: ["Tester","Designer"]}}, {"age": {$in: [41,32]}}]}).pretty()

# Distinct() Method

- The **distinct()** method retrieves all unique values of a specified field from a collection and returns them as an array.
- **Syntax :** *db.Collection_name.distinct(field : <string>)*

- **Example:**
  *db.students.distinct( "city")*

# Sort() Method

- The **sort()** method defines the order in which documents are returned from a collection based on the specified query criteria.
- This method must be applied to the query cursor before fetching any documents from the database.
- It accepts a document as a parameter, where each field–value pair specifies the sorting order for the result set.
- The value can be **1** for ascending order or **-1** for descending order sorting.
- **Syntax:** *db.Collection_Name.sort({field_name:1 or -1})*

# Sort() Method

**Examples:**

- db.students.find().sort({“job_title”:1})
- db.students.find({}, {"first_name":1, "last_name":1, "city":1}).sort({"job_title":1})
- db.students.find().sort({“course":1, "city": 1}).pretty()
- db.students.find().sort({" course ":1, "city": -1}).pretty()
- db.students.find({"city":"Anand"}).sort({"couurse":1}).pretty()

# limit() Method

- The **limit()** method restricts the number of documents returned in a query result.
- It sets the maximum number of records or documents to be displayed in the output.
- **Syntax:** *db.collectionName.find(<query>).limit(<number>)*
- **Examples:**

  db.students.find().limit(5).pretty()

  db.students.find({"city":"Aannd"}).limit(5).pretty()

# skip() Method

- The **skip()** method is used to bypass the first ***n*** number of documents in the query result.
- You need to specify the number of records or documents to skip in the query result.
- **Syntax:** *db.collectionName.find(<query>).skip(<offset>)*

**Examples**

db.students.find().skip(5).pretty()

db.students.find({"city":"Aannd"}).skip(2).pretty()

db.students.find({"city":"Aannd"}).skip(2).limit(2)pretty()

# More methods

- The **findOne()** method retrieves and returns a single document that matches the specified query criteria.
- If multiple documents match the query criteria, the findOne() method returns the first document based on the natural order of documents as stored on disk.
- If no document matches the query, the method returns **null** as the result.
- **Syntax:** *db.Collection_Name.findOne(query:<document>)*
- **Examples:**

db.students.findOne()

db.students.findOne({"city":"Anand"})

db.students.findOne({"city":"Anand"}, {“first_name”:1,”last_name”:1})

# More methods

- The **findOneAndUpdate()** method updates the first document in the collection that matches the specified query criteria.
- If multiple documents match the selection criteria, only the first matching document is updated, while the **_id** field remains unchanged.
- **Syntax:** *db.collection. findOneAndUpdate(*

  *<selection_criteria:<document>, update_data: <document>)*
- **Example**

```
db.stud.findOneAndUpdate(
    {"city":"surat"},
    {$set: {"last_name": "shah"}})
```

# More methods

- The **findOneAndReplace()** method replaces the first document that matches the specified query criteria.
- It returns the original document if a match is found; otherwise, it returns **null.**
- **Syntax:** *db.collection.findOneAndReplace(*

  *<selection_criteria:<document>, replacement: <document>)*
- **Example**

```
db.stud.findOneAndReplace(
    {"first_name":"Anna"},
    {"first_name":"Ananya","last_name":"Khan","city":"Surat"})
```

# More methods

- The **findOneAndDelete()** method removes a single document from the collection that matches the specified query criteria.
- It deletes the first document in the collection that matches the specified filter query.
- **Syntax :** *db.collection. findOneAndDelete(<filter>)*
- **Example**

  db.stud.findOneAndDelete({"last_name":"khan"})

# Objectives

- **In this session, you will learn to....**
  - Working with indexes

# Create Index

- Indexes are specialized data structures that enable faster retrieval of data from documents.
- They are organized based on the values of the fields defined in the index.
- The **createIndex()** method is used to create one or more indexes on a collection.
- **Syntax:**

  *db.collection_name.createIndex( keys: { field_name: 1/ -1 ....} )*

  *db.collection_name.createIndexes( [keys: { field_name: 1/ -1 ...} ] )*

  *db.collection_name.dropIndex(<index_name>...)*

  *db.collection_name.dropIndexes()*

  *db.collection_name.getIndexes()*

# Create Index ( Examples )

- Create an index on first_name fields on collection.
  db.stud.createIndex({"first_name":1})
- To display an existing indexes from collection.
  db.stud.getIndexes()
- To remove the named index from collection
  db.stud.dropIndex("first_name_1")
- To remove all the indexes from collection
  db.stud.dropIndexes()
- Create an index on multiple fields
  db.students.createIndex({"first_name":1,"city":1})

# Create Index ( Examples )

- Create an index on multiple fields ( -1 denotes in ascending order).
  db.students.createIndex({"first_name":1,"city": **-1**})
- Create a unique index on rollno field.
  db.marks.createIndex({"rollno":1}, {**unique: true**})
- Create an index on multiples fields ( name property is used to specify name of the index)
  db.marks.createIndex({"rollno":1, "fname":1}, {"**name": "rollno#name**"})
- To remove multiple indexes from collection.
  db.marks.dropIndexes("rollno_1","**roll#name**")

# What is Aggregation ?

- Aggregation is used to carry out complex data processing and computations on collections of documents.
- It allows **grouping, filtering,** and **transforming** data to generate summarized results.
- Aggregation is typically performed using a **pipeline**, a framework for data aggregation modeled on the concept of data processing pipelines.
- A **pipeline** in MongoDB aggregation is a series of stages that process documents step by step to generate meaningful and summarized data.
- Each stage of the pipeline transforms the documents as they pass through it and allows for operations like filtering, grouping, sorting, reshaping and performing calculations on the data.

# Working with Aggregation

Aggregation operations can be performed in two ways:

1. **Using single purpose aggregation method.**
2. **Using Aggregation Pipeline.**

# Working with Aggregation

**1. Using single purpose aggregation methods:**

- db.collection.estimatedDocumentCount(),
- db.collection.count() and
- db.collection.distinct().

**Example**

```
db.employee.aggregate([
    { $group: { _id: "$deptno", totalemps: { $sum: 1 } } }
])
```

**Group** the documents by ***deptno*** and then count the total number of employees in each department.

# Working with Aggregation

## 2. Using Aggregation Pipeline :

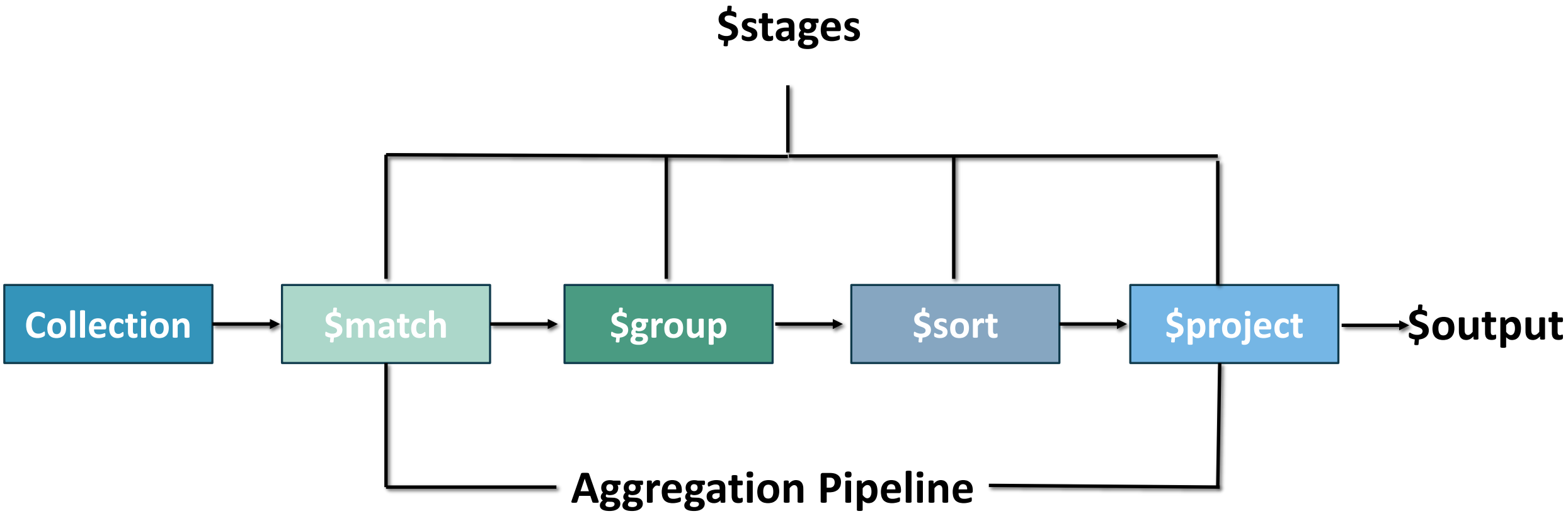
- The aggregation pipeline is an array of one or more stages passed in the **db.aggregate()** or **db.collection.aggregate()** method.
- db.collection.aggregate([ {stage1}, {stage2}, {stage3}...])
- Each stage takes the **output** from the **previous stage**, processes it, and passes it as input to the next **stage.**
- The aggregation framework processes a pipeline of stages on the collection data and produces the output in the desired format.
- A group key can be a single field, a combination of fields, or the result of an expression.

# When to use aggregation

- Grouping and calculating statistics
- Complex data transformations
- Multi-stage processing
- Computed fields
- Joining collections ($lookup)

# Working with Aggregation

## 2. Using Aggregation Pipeline :



# Aggregation Stages

**$match -** It selects only the documents that match the specified criteria from a collection.

**Example**

```
db.employees.aggregate( [
    { $match: { deptno: 10 } } ,
    { $limit: 5 }
] )

db.products.aggregate([
    { $match: { price: { $lt: 100 } } }
])
```

# Aggregation Stages

## $group

- It groups the input documents by the specified _id expression and returns a single document with the accumulated values for each distinct group.

**Example**

```
db.orders.aggregate([
    { $group: {
        _id: "$orderid",
        totalSales: { $sum: "$amount" }
    }}
])
```

# Aggregation Stages

## $group ( more examples )

```
db.employees.aggregate([
   { $match:{ gender:'male'} },
   { $group:{ _id: "$deptno", totaemployees: { $sum:1 } }
}])
```

```
db.employees.aggregate([
   { $match:{ gender:"female"} },
   { $group:{ _id:{ deptno: $deptno,
         totalsal: { $sum: "$salary"} , avgsal: { $avg: "$salary" } }
}])
```

# Aggregation Pipeline Flow

Input – All documents in collection

↓

Stage-1: **$match** ( filter )

↓

Stage-2: **$project** (reshape)

↓

Stage 3: **$sort** (order)

↓

Stage 2: **$limit** (reshape)

↓

Output: **Final result set**

# Pipeline Stages Process

```
db.orders.aggregate( [
    { $match: { status: "completed" } },      // Stage 1: Filter
    { $group: {                               // Stage 2: Group
        _id: "$orderid",
        total: { $sum: "$total" }
    }},
    { $sort: { total: -1 } },                 // Stage 3: Sort
    { $limit: 10 }                            // Stage 4: Limit
])ss
```

# Aggregation Stages

## $sort

- It sorts the documents based on a specified field in either ascending or descending order.

**Example**

```
db.employees.aggregate([ { $match:{ gender:'male'}},
        { $sort:{ ename: 1}}
])

db.employees.aggregate([ { $match:{ gender:'male'}},
   { $group:{ _id:{ deptno:'$ deptno '}, total: { $sum:1} } },
         { $sort:{ deptno:1}}])
```

# Aggregation Stages

**$project -** $project is an aggregation stage in MongoDB that reshapes the documents in a collection.

It allows you to:

- Include or exclude specific fields in the output.
- Add new fields or computed fields based on existing data.
- Rename fields for clarity.
- Perform simple expressions or calculations to create new fields.

**Example**

```
db.universities.aggregate([
    { $project : { _id : 0, empno: 1, ename: 1, salary : 1 } }
]).pretty()
```

# Aggregation Stages

```
db. universities.aggregate([
  { $project: { employeeName: "$ename", department: 1 } }])

db.employees.aggregate([
  { $project: { employeeName: "$name", annual: { $multiply: ["$salary", 12] },
     department: 1,  _id: 0 }} ])

db.universities.aggregate([
     { $project : { _id : 0, empno: 1, ename: 1, salary : 1 } }
]).pretty()
```

# Working with Aggregation ($sum)

**Use function based on multiple group ( Fields )**

```
db.emps.aggregate([
    {$group : {_id: {department:"$department", job_title:"$job_title"},
        count:{$count:1}}} ])
```

**Use multiple functions based on multiple group ( Fields )**

```
db.emps.aggregate([{
    $group : {_id:{department:"$department", job_title:"$job_title"},
    count:{$sum:1},max:{$max: "$salary"}}}])
```

More group functions are ***$bottom, $bottomN, $first, $firstN, $last, $lastN, $top, $topN, $max, $maxN*** and many more….

# Arithmetic Operators

## $add

- $add operator is used to sum multiple numbers and can accept any number of arguments.
- It is commonly used in the **$project, $group,** and **$addFields** stages of an aggregation pipeline to combine multiple expressions and create new fields or values.
- **Syntax :** *{ $add: [ <expression1>, <expression2>, … ] }*

**Example:**

```
db.items.aggregate(
    [{$project: {"itemcd": 1, "iname": 1, "sprice":1, "profit":1,
    SalePrice: {$add: ["$sprice","$profit"]}}}
])
```

# Arithmetic Operators

## $subtract

- $subtract calculates the difference between two numeric values and returns the result.
- It is commonly used in the **$project, $group,** and **$addFields** stages.
- **Syntax :** { $subtract : [ <expression1>, <expression2>, ... ] }

```
db.workinghrs.aggregate([ {
    $project: {
      _id: "$empno",  month: "$month",
     total: { $multiply: ["$totalhrs", "$wages"] } } },
     { $addFields: {  taxvalue: { $subtract: ["$total", 1000] }  } }])
```

# Arithmetic Operators

## $divide

- $divide operator is used to perform division operations.
- It is frequently used in the **$project, $group,** and **$addFields** stages of an aggregation pipeline.
- **Syntax :** { $divide : [ <expression1>, <expression2>, ... ] }

```
db.empwork.aggregate([ {
    $project: {
        name: 1,
        hourly_salary: { $divide: ["$salary", "$hours_worked"]}}}
])
```

# Arithmetic Operators

## $multiply

- $multiply is used to calculate the product of two numbers in an expression and return the result.
- It can be used in the aggregation pipeline.
- **Syntax :** { $multiply : [ <expression1>, <expression2>, ... ] }

**Example:**

```
db.workinghrs.aggregate([
    {$project:
        { totalValue: { $multiply: ["$totalhrs", "$wages"] }
    } }
] )
```

# $lookup

- $multiply is used to perform a join-like operation between two collections.
- **Example:**

**Users collection:**

{ _id: 1, name: "John", age: 25 }

{ _id: 2, name: "Sara", age: 26 }

**Order collection:**

{ _id: 101, product: "Laptop", userId: 1 }

{ _id: 102, product: "Phone", userId: 1 }

{ _id: 103, product: "Tablet", userId: 2 }

# $lookup - continue

**JOIN users → orders based on _id & userId**

```
db.users.aggregate([
    {
        $lookup: {
            from: "orders",
            localField: "_id",
            foreignField: "userId",
            as: "userOrders"
    }
}]).pretty();
```

```
{
    "_id": 1, "name": "John", "age": 25,
    "userOrders": [
        {"_id": 101, "product": "Laptop",
            "userId": 1 },
        {"_id": 102, "product": "Phone",
            "userId": 1 }
 ]
}
```

# $lookup - continue

**JOIN users → orders based on _id & userId ( Project Example )**

```
db.users.aggregate([
 {
    $lookup: {
        from: "orders", localField: "_id", foreignField: "userId",     as: "orders"
    }
}.
{
   $project: {  name: 1,  "orders.product": 1 } }
]).pretty();
```

Q&
A